Software Design Document

Project Title: Vocal-Dine AI Powered Voice-Interactive Smart Menu System

Submitted by:

Alishba Abbas

Roll No: S25BINFT7M04035

Submitted to:

Dr Professor Dost Muhammad khan

Supervisor, Department of Information Technology

Department of Information Technology

Faculty of Computing

THE ISLAMIA UNIVERSITY OF BAHAWALPUR

Academic Year 2025-2027

Summary

This Software Design Document (SDD) outlines the technical and architectural design for Vocal-Dine, an AI-powered voice-interactive smart menu system tailored for the hospitality industry. Vocal-Dine modernizes the dining experience by replacing static, printed or digital menus with an accessible, conversational web interface. By scanning a dynamic QR code, customers can explore menus, filter choices, receive intelligent, context-aware food recommendations, and place their orders entirely through natural voice commands.

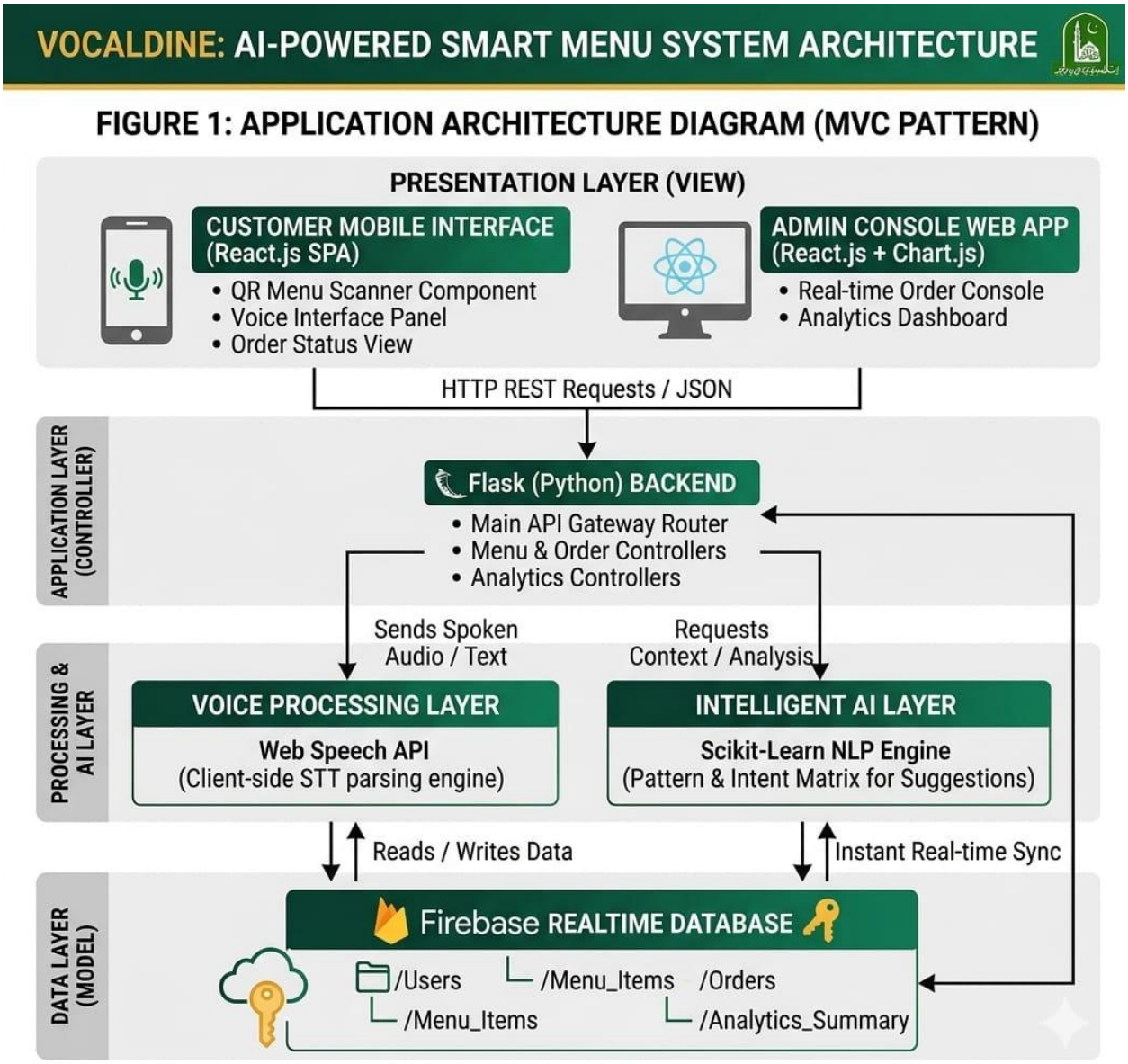
On the administrative end, the system provides restaurant management with a real-time order processing panel and an analytics dashboard to track operational efficiency and sales trends. This document delivers a deep dive into Vocal-Dine's multi-layered application architecture, relational data schemas, secure backend design logic, user interface philosophies, and comprehensive UML representations to establish a clear structural baseline for complete system implementation.

Table of Contents

1. Application Architecture

1.1 Architectural Pattern Overview

VocalDine is built using a decoupled, multi-layered architecture that aligns with the Model-View-Controller (MVC) architectural pattern to ensure loose coupling, high maintainability, and clean separation of concerns.



The system is cleanly divided into four distinct operational layers:

- **Presentation Layer (View):** Built with **React.js**, delivering a single-page application (SPA) experience for both customers and admins. It consumes REST APIs and manages user state efficiently.
- **Application Layer (Controller):** Developed using **Python** (Flask), serving as the central engine processing incoming text requests, triggering internal NLP pipelines, routing business logic, and structuring JSON responses.
- **Voice Processing Layer:** Leverages the native client-side browser Web Speech API for real-time speech-to-text (STT) parsing, sending text payloads directly to the controller layer.
- **Data Layer (Model):** Powered by **Firebase** Realtime Database, which structures application data objects and synchronizes changes instantaneously between customer actions and the admin panel.

1.2 Component Breakdown

- **Models (Data Structure):** Handled as structured JSON documents and collections inside Firebase, representing core system definitions: *User, Menu_Item, Order, and Analytics_Summary.*
- **Views (User Interface):** Componentized React.js interfaces that receive state updates and render clean layouts. These include the *QRMenuScanner, VoiceInteractionPanel, OrderSummaryView, AdminOrderConsole, and SalesAnalyticsDashboard.*
- **Controllers (Logical Routers):** Flask-based API blueprints that catch client actions, run analytical logic, change states in Firebase, and output structured payload data.

1.3 Controller Logic & Functions

The Flask application architecture maps explicit endpoints to clean execution hooks:

MenuController

- **get_menu_by_qr(qr_id)**: Decodes scanned restaurant reference tokens and pulls matching food menu nodes.
- **filter_menu_by_voice(extracted_intent):** Matches voice search intents (e.g., "show burgers") against food item titles, descriptions, and tag attributes.

VoiceNLPController

- **process_voice_intent(text_payload):** Runs text extracted via client-side speech processing through custom intent-matching patterns to identify structural parameters like ACTION (order, view, recommend) and ITEM_NAME.
- **generate_ai_recommendations(current_cart, user_history):** Utilizing an analytical scoring wrapper, it processes selected items to generate complementary food suggestions (e.g., matching a drink with a burger).

OrderController

- **create_voice_order(order_payload):** Validates item availability, compiles item breakdowns, sets states to Pending, and posts objects to Firebase.
- **update_order_status(order_id, status_code):** Accessible only by authenticated admin tokens; updates order status records (e.g., Accepted, Preparing, Ready).

AdminAnalyticsController

- **fetch_dashboard_insights():** Compiles transaction flows and high-demand item analytics arrays for structured visualization generation.

1.4 API & External Library Integration

- **Web Speech API (Speech Recognition):** Running on the client browser browser environment, it captures streaming microphone audio, processes acoustics locally via built-in engine parameters, and passes instant text conversions to React component hooks.
- **Scikit-Learn NLP Integration:** Executed server-side within the Flask pipeline to compute command vectors, matching phonetic and semantic user inputs into strict executable ordering entities.
- **Chart.js Dependency:** Initialized within the React Admin dashboard to fetch transactional arrays and map them into responsive visual data grids and charts.

1.5 System Coding Conventions

- **Frontend (React.js):** Follows ECMAScript 6 (ES6) specifications. Features descriptive camelCase naming for variables/functions (e.g., isMicrophoneActive) and PascalCase for components (e.g., VoiceButton). Uses strict functional hooks for clean code reusability.

- **Backend (Flask):** Adheres completely to PEP 8 style formatting. Utilizes lowercase snake_case for routes, controller functions, and utility engines (e.g., `calculate_total_bill()`).
- **Data Payloads:** All client-server request/response communication schemas rely entirely on standardized JSON patterns.

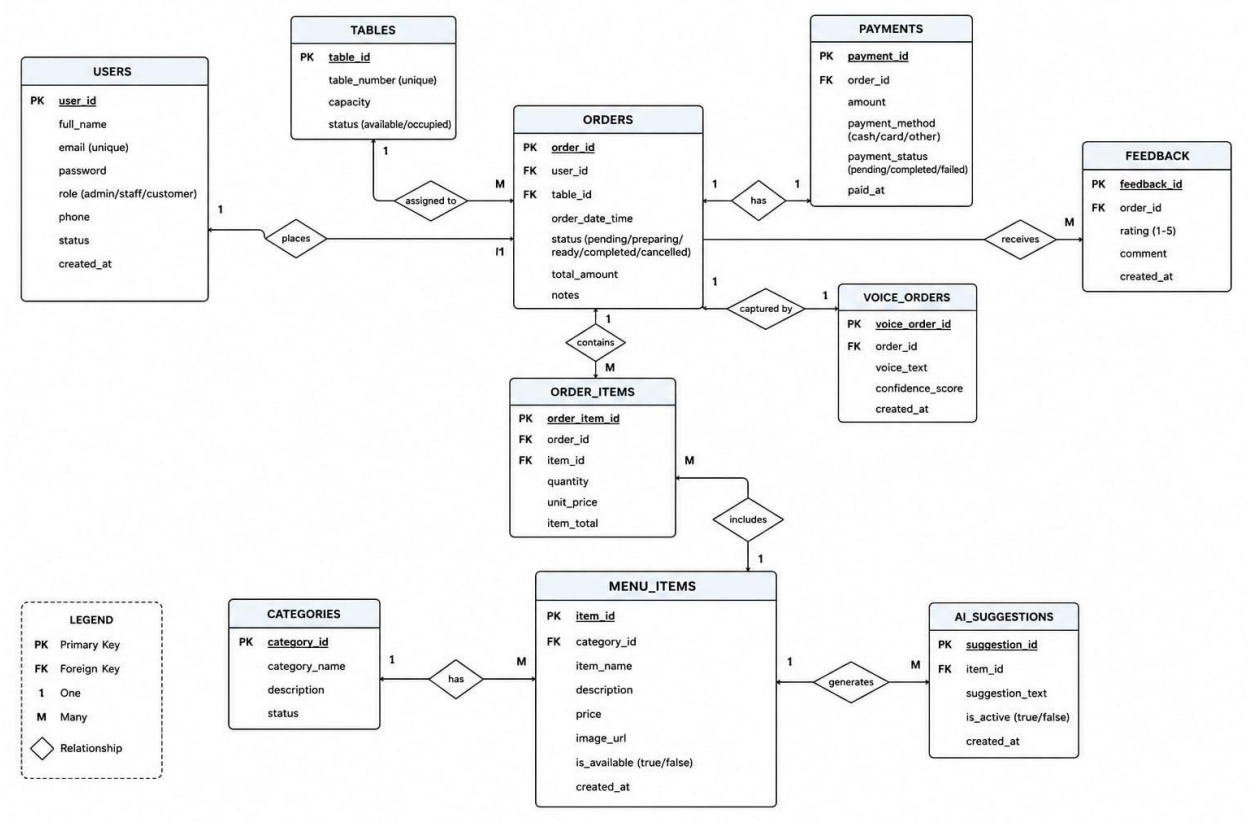
2. Data Model Schema

2.1 Conceptual Schema Description

VocalDine implements a document-centric, relational JSON schema mapping inside Firebase to achieve horizontal data scaling and ultra-low synchronization latency. When customers place an item inside their digital cart or trigger order processing, database hooks directly alert listening administrative clients.

The entity logic relies on a central restaurant structural hub: `Menu-Items` contain detailed property profiles, which are queried by `Orders`. Each order documents operational steps and relates structurally to an `Admin` entity for fulfillment tracking.

2.2 Entity-Relationship (ER) Diagram



2.3 Data Dictionary & Entity Properties

Entity: Menu_Item

Field Name	Data Type	Key Type	Input Validation Constraints
Item_id	String	Primary Key	Autogenerated unique alphanumeric string
name	String	Normal Field	Alphanumeric characters only; cannot be null
price	Float	Normal Field	Must be a positive numeric value > 0
category	String	Normal Field	Enforced selection (e.g., Appetizer, Main, Drink)
is_available	Boolean	Normal Field	Default set to True

Entity: Order

Field Name	Data Type	Key Type	Input Validation Constraints
order_id	String	Primary Key	Autogenerated unique alphanumeric string
table_number	Integer	Normal Field	Numeric limits mapped between 1 and 50

Timestamp	String / Long	Normal Field	Standardized ISO 8601 server datetime format
items_ordered	Array (JSON)	Foreign Entity	Array contain item objects mapping item_id & quantity
total_amount	Float	Normal Field	Must match calculated mathematical sum of items
order_status	String	Normal Field	Value restricted to: Pending, Accepted, Ready

Entity: Admin

Field Name	Data Type	Key Type	Input Validation Constraints
admin_id	String	Primary Key	Autogenerated identity hash string
username	String	Normal Field	Alphanumeric values; minimum 5 characters

password_hash	String	Normal Field	Encrypted password string string format

3. User Interface Design

3.1 Design Philosophy, Color Scheme, & Fonts

VocalDine features a modern, clean, and highly intuitive UI/UX design tailored specifically for fast-paced hospitality environments.

- **Design Philosophy:** Minimalist component styling focused heavily on structural visibility, readability, and prominent accessibility controls for smooth, hands-free voice-command interactions.
- **Color Scheme:** A modern and professional color palette will be used to ensure good readability and user experience. The final color scheme will be selected during the UI design phase. We choose colors that make the food look good and keep the screen easy to read for everyone. The design is modern, and we have kept it flexible so we can easily update the colors later if needed. It will look professional on both mobile phones and computer screens.
- **Typography & Fonts:** The layout uses the clean sans-serif typeface Inter or Poppins to ensure exceptional text readability on mobile devices under various lighting conditions.

3.2 Responsive Screen Layouts & Constraints

The system is built entirely around a mobile-first, highly responsive web layout.

- **Customer Interface:** Optimized exclusively for standard iOS and Android smartphone display constraints, ensuring full accessibility directly inside web view screens after scanning a QR code without requiring a separate app installation.
- **Admin Interface:** Tailored for horizontal tablet, laptop, and desktop view configurations, utilizing dynamic flexbox components to resize order processing lists and live metric graphs fluidly.

3.3 Screen Lists, Control Elements, & Input Constraints

1. Customer Welcome & QR Landing Screen

- **Functional Purpose:** Directs the customer into the menu application context once a table QR code is verified.
- **UI Controls & Elements:** Prominent animated Scan Success Banner, table number display badge, and a large "Explore Menu" Call-to-Action Button.

2. Voice-Interactive Smart Menu Grid

- **Functional Purpose:** The primary interface where users browse food categories, review real-time recommendations, and speak commands.
- **UI Controls & Elements:**
  - Floating Microphone Activation Button (turns glowing red when actively listening).
  - Continuous Live Transcript Preview Bar that converts voice input to text in real-time.
  - Food Item Cards featuring descriptive pictures, pricing tags, and a manual "Add to Cart" Button.
  - Dynamic AI Recommendation Panel displaying matching suggestions.
- **Input Constraints:** Voice requests timeout after 6 seconds of absolute silence to prevent empty payload processing. Unclear spoken tokens trigger a soft fallback alert asking the user to repeat.

3. Voice Order Confirmation & Status Screen

- **Functional Purpose:** Displays compiled item totals and tracks live prep updates.
- **UI Controls & Elements:** Itemized bill overview grid, an interactive "Confirm Order via Voice" Toggle, and an animated horizontal ordering status tracker.

4. Admin Order Processing Dashboard

- **Functional Purpose:** Allows kitchen staff and administrators to review, prioritize, and manage live incoming restaurant queues.
- **UI Controls & Elements:**
  - Split-screen master-detail arrangement organizing incoming orders by chronological urgency.
  - "Accept Order" Button (Green) and "Reject Order" Button (Red).

- Status Update Dropdown Field (Pending $\rightarrow$ Preparing $\rightarrow$ Ready).

5. Admin Sales Analytics Console

- **Functional Purpose:** Displays daily revenue performance metrics and highlights popular food selections.
- **UI Controls & Elements:** Interlinked Date-Range Selector Fields, total revenue numeric summary blocks, and dynamic Chart.js Canvas Components displaying trending orders.
- **Input Constraints:** Form date validation prevents end-date bounds from referencing values older than start-date entries.